

Agrégation externe d'informatique — Épreuve blanche

Arbres de Merkle et vérification légère dans Bitcoin

Théorie, simulations et implémentations — durée indicative : 4 h

Cette épreuve alterne des **questions théoriques** (à démontrer), des **tâches d'implémentation** (à programmer en Python 3, modules `hashlib`, `numpy`) et des **expériences de simulation** (à mener et à commenter). **Aucune réponse n'est fournie.** On attend un code lisible, testé, et une analyse (correction, complexité, sécurité, convergence). Les signatures données fixent seulement les interfaces attendues.

1 Cadre : blocs, transactions et en-tête

Une blockchain de type Bitcoin est une suite de *blocs*. Chaque bloc contient une liste ordonnée de *transactions* $t_0, \dots, t_{n-1}$ et un *en-tête* de taille fixe. L'en-tête comporte notamment : le haché de l'en-tête précédent, un *condensé* des transactions appelé *racine de Merkle*, un horodatage, une *cible* de difficulté et un entier libre, le *nonce*. La sécurité repose sur une fonction de hachage. Bitcoin utilise le *double SHA-256*

$$H(m) = \text{SHA256}(\text{SHA256}(m)) \in \{0, 1\}^{256},$$

que l'on manipulera sous forme d'objets `bytes` de longueur 32. On identifie une transaction t_i à son *identifiant* $\text{txid}_i = H(t_i)$. Toutes les concaténations $a||b$ portent sur les octets bruts.

Objectifs. On cherchera systématiquement à : (i) *démontrer* les propriétés annoncées ; (ii) *implémenter* les fonctions spécifiées et les *tester* ; (iii) *simuler* pour confronter la théorie à l'expérience. Les six parties sont largement indépendantes ; on pourra les aborder dans l'ordre de son choix.

2 Fonction de hachage : propriétés et collisions

```
def H(m: bytes) -> bytes:
    """Double SHA-256 : renvoie 32 octets. À implémenter avec hashlib."""
    ...
```

Q2.1 (implémentation) Implémenter `H`. Écrire un test vérifiant le déterminisme ($H(m) = H(m)$) et l'effet d'avalanche (un bit modifié change massivement la sortie, mesuré par la distance de Hamming moyenne sur 32 octets).

Q2.2 (théorie) Montrer qu'une fonction $\{0, 1\}^* \rightarrow \{0, 1\}^{256}$ ne peut être injective, et donc que des collisions existent. Expliquer en quoi la *résistance aux collisions* est une hypothèse *calculatoire* et non un théorème.

Q2.3 (théorie) On modélise H comme un tirage uniforme dans un ensemble à $M = 2^b$ valeurs. On tire k valeurs indépendantes. Montrer que la probabilité d'absence de collision vérifie $q_k = \prod_{i=0}^{k-1} (1 - i/M) \leq \exp(-k(k-1)/2M)$, et en déduire que le seuil de collision probable est de l'ordre de $\sqrt{M} = 2^{b/2}$. Commenter le choix $b = 256$.

Q2.4 (simulation) Pour un hachage *tronqué* à $b \in \{16, 24, 32\}$ bits (ne garder que les b premiers bits de H), estimer par une méthode de Monte-Carlo le nombre moyen de tirages avant la première collision, et confronter la mesure à la prédiction $\Theta(2^{b/2})$. Présenter les résultats dans un tableau ou un graphique.

Q2.5 (théorie) Distinguer trois notions de sécurité d'une fonction de hachage : *résistance à la préimage*, à la *seconde préimage*, et *aux collisions*. Ordonner ces propriétés (laquelle implique laquelle ?) et préciser laquelle est requise pour la sécurité des preuves de Merkle de la partie 4.

3 Racine de Merkle d'un bloc

La *racine de Merkle* résume les n transactions. On procède par niveaux : partant de la liste des txid_i , on remplace chaque paire consécutive (a, b) par $H(a||b)$; si un niveau contient un nombre *impair* d'éléments, on **duplique son dernier élément** avant l'appariement (règle de Bitcoin). On s'arrête sur un unique haché, la racine.

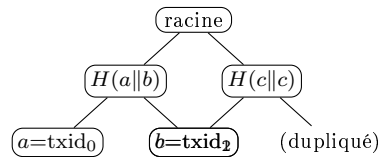


Figure 1 — bloc à 3 transactions : le dernier haché est dupliqué.

```

def merkle_root(txids: list[bytes]) -> bytes:
    """Racine de Merkle (regle Bitcoin : duplication si niveau impair).
    Precondition : len(txids) >= 1."""
    ...
  
```

Q3.1 (implémentation) Implémenter `merkle_root`. Traiter correctement le cas $n = 1$ et la duplication. Tester sur des blocs de 1, 2, 3, 4, 5 transactions.

Q3.2 (théorie) Soit n le nombre de transactions. Montrer que le calcul de la racine effectue $O(n)$ évaluations de H et que la hauteur de l'arbre vaut $\lceil \log_2 n \rceil$. On majorera la somme des tailles des niveaux successifs.

Q3.3 (théorie) Décrire une conséquence de la règle de duplication en termes de sécurité : montrer qu'un bloc à nombre impair de transactions et un bloc obtenu en dupliquant réellement la dernière transaction peuvent produire la *même* racine (malléabilité de type « CVE-2012-2459 »). Que faut-il vérifier en plus de la racine ?

4 Preuves de Merkle et vérification simplifiée (SPV)

Un client *léger* (*Simplified Payment Verification*) ne stocke que les en-têtes des blocs. Pour lui prouver qu'une transaction t appartient à un bloc de racine R , on fournit une *preuve de Merkle* : la liste des hachés frères rencontrés en remontant de la feuille $H(t)$ jusqu'à la racine, chacun muni de son côté ('L' ou 'R').

```

def preuve_merkle(txids: list[bytes], i: int) -> list[tuple[bytes, str]]:
    """Chemin de freres (haché, cote) de la feuille i jusqu'a la racine."""
    ...
def verifier_preuve(feuille: bytes, preuve, racine: bytes) -> bool:
    """Recalcule la racine a partir de la feuille et de la preuve."""
    ...
  
```

Q4.1 (implémentation) Implémenter `preuve_merkle` et `verifier_preuve`. Vérifier, sur un bloc aléatoire de n transactions, que toute transaction présente est acceptée, et qu'une transaction absente (ou une preuve altérée) est rejetée.

Q4.2 (théorie) Montrer que la taille d'une preuve d'inclusion est $O(\log n)$. Comparer, pour $n = 10^4$ transactions, au coût de télécharger le bloc entier ; justifier l'intérêt du modèle SPV.

Q4.3 (théorie) Sécurité (soundness). Démontrer, par récurrence sur la hauteur de l'arbre et sous l'hypothèse de résistance aux collisions de H , que si `verifier_preuve` accepte une feuille pour la racine authentique R , alors — sauf à exhiber une collision de H — cette feuille figure réellement parmi les transactions du bloc. Expliciter la *réduction* : comment une fausse preuve fournit-elle algorithmiquement une collision ?

Q4.4 (simulation) Mesurer empiriquement la taille moyenne des preuves (nombre de frères) en fonction de n pour $n = 2^j$, $j = 1, \dots, 14$, et confronter à $\lceil \log_2 n \rceil$. Tracer le résultat.

Q4.5 (implémentation) Un message `merkleblock` de Bitcoin transmet, en une seule fois, les preuves d'inclusion de *plusieurs* transactions d'intérêt à l'aide d'un *arbre de Merkle partiel* (les sous-arbres ne contenant aucune transaction d'intérêt sont remplacés par leur seul haché racine). Spécifier puis implémenter une fonction `arbre_partiel(txids, indices)` qui produit un tel certificat compact, et `verifier_partiel` qui le valide. Comparer sa taille à la somme des preuves individuelles.

5 Preuve de travail (minage)

L'en-tête e contient un *nonce* ν . *Miner* le bloc, c'est trouver ν tel que $H(e_\nu)$, interprété comme un entier sur 256 bits, soit strictement inférieur à une *cible* T . On note $p = T/2^{256}$ la probabilité, pour un essai, de satisfaire la condition (modèle de l'oracle aléatoire).

```
def mine(entete_sans_nonce: bytes, cible_bits_zero: int) -> tuple[int, bytes]:
    """Cherche le plus petit nonce (32 bits, big-endian concatene) tel que
       H(entete || nonce) commence par 'cible_bits_zero' bits a zero.
       Renvoie (nonce, hache). Ici la cible T = 2**(256 - cible_bits_zero)."""
    ...
```

Q5.1 (implémentation) Implémenter `mine` en exigeant k bits de tête nuls (donc $p = 2^{-k}$). La tester pour k croissant et relever le nombre d'essais.

Q5.2 (théorie) Soit N le nombre d'essais jusqu'au premier succès. Donner la loi de N , montrer que $\mathbb{E}[N] = 1/p = 2^k$ et que le minage *termine presque sûrement* ($\mathbb{P}(N = \infty) = 0$). Ce processus est-il un algorithme au sens usuel ? Discuter la notion de *terminaison probabiliste*.

Q5.3 (simulation) Pour $k \in \{8, 12, 16, 20\}$, estimer $\mathbb{E}[N]$ par Monte-Carlo (moyenne sur plusieurs minages indépendants) et comparer à 2^k . Tracer l'histogramme empirique de N pour un k fixé et vérifier qu'il épouse une loi géométrique (décroissance de raison $1 - p$).

Q5.4 (théorie & simulation) Bitcoin ajuste périodiquement la cible pour maintenir un temps moyen entre blocs constant, malgré une puissance de calcul λ (hachages par seconde) variable. En supposant N géométrique et λ constant sur la période, exprimer le temps moyen d'un bloc en fonction de p et λ , puis proposer une règle d'ajustement de T lorsque λ double. Illustrer par une simulation où λ varie par paliers.

Q5.5 (théorie & simulation) *Course entre chaînes*. Un attaquant disposant d'une fraction $\alpha < 1/2$ de la puissance de calcul tente de rattraper la chaîne honnête partie avec z blocs d'avance. En modélisant l'écart de blocs par une marche aléatoire, estimer par simulation la probabilité de rattrapage en fonction de α et z , et observer sa décroissance exponentielle en z . Relier au principe de *confirmation* (attendre z blocs avant de considérer une transaction comme définitive).

Prolongements (facultatifs)

- Comparer l'arbre de Merkle binaire de Bitcoin à l'arbre de *Patricia-Merkle* d'Ethereum et aux *arbres de Verkle* (taille de preuve $O(1)$).
- Étudier la complexité et la sécurité d'une mise à jour dynamique de la racine lorsqu'une transaction change ($O(\log n)$).
- Relier la résistance aux collisions aux *fonctions à sens unique* et au cadre de la cryptographie à sécurité prouvée.

Références

1. S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008 (sections « Merkle Tree » et « Calculations »).
2. R. C. Merkle, *A digital signature based on a conventional encryption function*, CRYPTO '87.
3. A. M. Antonopoulos, *Mastering Bitcoin*, O'Reilly ; A. M. Antonopoulos, G. Wood, *Mastering Ethereum*, O'Reilly.
4. J. Katz, Y. Lindell, *Introduction to Modern Cryptography*, CRC Press.

Épreuve blanche (énoncé seul, sans corrigé) — Pr Agrégé El Hadiq Zouhair, cpgeacademy.org.